Laboration 5 - DOM, events och formulärhantering

Saknad

10/11/2026

Försök 1

Pågår

NÄSTA: Lämna in uppgift

Lägg till kommentar

Unlimited Attempts Allowed

11/1/2026

Information

Syfte/mål

Syftet med uppgiften är att:

- Förstå hur JavaScript kan användas för att läsa och förändra innehåll i DOM.
- Använda event och eventlyssnare för att skapa interaktivitet på en webbsida.
- Läsa och hantera värden från HTML-formulär.
- Validera användarinmatning och visa tydliga felmeddelanden.
- Skapa och lägga till HTML-element med JavaScript.
- Använda `localStorage` för att lagra information i webbläsaren.
- Använda `JSON.stringify()` och `JSON.parse()` för att lagra och hämta strukturerad data.
- Strukturera JavaScript-kod i mindre funktioner med tydliga ansvarsområden.
- Använda Git och GitHub för att versionshantera programkod.

I laborationen bygger du vidare på de kunskaper om bland annat funktioner, arrayer och objekt som du arbetade med i föregående moment. Nu ska dessa användas för att skapa en interaktiv webbapplikation.

Teori och läsanvisningar

För att förbereda dig inför uppgiften, ta del av föreläsningar, teori och läsanvisningar samt eventuella övningar och quizar i modulens avsnitt på kurswebbplatsen. Prova också att experimentera med kodexempel i din utvecklingsmiljö för att stärka din förståelse innan du sätter igång med uppgiften.

Prova gärna att köra de kodexempel som finns i teorin själv och experimentera med att ändra värden och kod. Att skriva och testa små mängder kod åt gången är en viktig del av att lära sig programmering.

Du kommer även att använda arrayer, objekt, loopar och funktioner från föregående moment.

Uppgiftsbeskrivning

I denna laboration ska du skapa funktionaliteten till en **studentkortsgenerator**.

Du får färdiga HTML- och CSS-filer som utgör gränssnittet till applikationen samt en påbörjad JavaScript-fil. Din uppgift är att färdigställa JavaScript-koden så att användaren kan skapa studentkort utifrån information som matas in i ett formulär.

När ett studentkort skapas ska informationen även sparas i en historik med hjälp av webbläsarens `localStorage`.

Applikationen ska kunna:

1. läsa information från formuläret
2. kontrollera att obligatoriska fält är ifyllda
3. visa tydliga felmeddelanden om något saknas
4. generera ett studentkort med den angivna informationen
5. använda det typsnitt användaren valt
6. spara skapade studentkort i en historik
7. visa tidigare sparade studentkort
8. uppdatera historiken direkt när ett nytt studentkort skapas
9. rensa formuläret och eventuella felmeddelanden
10. radera hela historiken

Demonstration

Nedan finns en film (*utan ljud*) som visar exempel på hur en färdig lösning kan fungera.

Arbetsfiler

Till laborationen finns **färdiga arbetsfiler (https://canvas.miun.se/courses/11718/files/822233?wrap=1)** , ladda ner dessa. Packa upp filerna på valfri plats på din dator.

Projektet innehåller bland annat:

```
index.html
css/
    styles.css
js/
    main.js
```

All programmering i laborationen ska göras i `main.js`. Du ska inte ändra HTML- eller CSS-filerna.

`main.js` innehåller en påbörjad struktur med DOM-referenser, variabler och förslag på funktioner som programmet kan delas upp i.

Du ska färdigställa funktionerna och skapa de eventlyssnare som behövs för att applikationen ska fungera.

Syftet med den givna strukturen är att hjälpa dig att dela upp ett större programmeringsproblem i mindre delar. Försök därför att använda den givna strukturen och låt varje funktion ha ett tydligt ansvarsområde.

Versionshantering med Git och GitHub

När du har laddat ner och packat upp arbetsfilerna ska du skapa ett **eget Git-repository** för laborationen och publicera detta på GitHub.

Du ska använda Git för att versionshantera projektet under **hela arbetet med laborationen** och göra regelbundna commits allt eftersom du utvecklar funktionaliteten.

Versionshistoriken ska tydligt visa hur lösningen har utvecklats över tid. Det är alltså inte tillräckligt att skapa en eller ett fåtal commits först när laborationen är färdig.

Skriv commit-meddelanden som kortfattat beskriver **vad du har gjort eller förändrat i koden**, exempelvis:

```
Lade till validering av formulärfält
Skapade funktion för att visa felmeddelanden
Implementerade generering av studentkort
Lade till lagring av historik i localStorage
Skapade utskrift av historik till DOM
Lade till funktion för att radera historik
```

Undvik generella commit-meddelanden som `ändringar`, `uppgift`, `fix` eller `klart`.

Studentkort

Formuläret innehåller fält för:

- namn
- e-postadress
- telefonnummer
- typsnitt

När användaren skickar formuläret ska JavaScript läsa de angivna värdena och, om valideringen lyckas, uppdatera studentkortet som visas på sidan.

Studentkortet ska visa:

- namn
- e-postadress
- telefonnummer

Det typsnitt som användaren valt i formuläret ska användas för informationen på studentkortet.

Validering och felmeddelanden

Innan ett studentkort skapas ska användarens inmatning kontrolleras.

Följande fält är obligatoriska:

- namn
- e-postadress
- telefonnummer

Det räcker att kontrollera att respektive fält innehåller ett värde, dvs att formuläret inte är tomt. Du behöver exempelvis inte kontrollera om en angiven e-postadress har ett korrekt format.

Om ett eller flera obligatoriska fält är tomma ska:

- ett tydligt felmeddelande skapas för varje fel
- samtliga felmeddelanden lagras i en array

- felmeddelandena skrivas ut som en lista (`ul`) på sidan

Om valideringen misslyckas ska **studentkortet inte uppdateras och ingen information ska sparas i historiken**.

Om användaren exempelvis lämnar både namn och telefonnummer tomma ska alltså två separata felmeddelanden visas som en punktlista.

Historik och Web Storage

Varje gång ett giltigt studentkort skapas ska informationen även sparas i en historik.

Varje studentkort i historiken ska representeras av ett **objekt**, och historiken ska hanteras som en **array med objekt**.

Varje objekt ska innehålla:

- namn
- e-postadress
- telefonnummer
- valt typsnitt

Historiken ska sparas i webbläsarens `localStorage`.

Eftersom `localStorage` lagrar information som text behöver historiken omvandlas när den sparas och hämtas. Använd:

- `JSON.stringify()` när historiken sparas
- `JSON.parse()` när historiken hämtas

När sidan öppnas ska eventuell tidigare historik hämtas från `localStorage` och visas på sidan. Historiken ska alltså finnas kvar även om användaren stänger eller laddar om webbsidan.

Uppdatering av historiken

När användaren skapar ett nytt giltigt studentkort ska:

1. studentkortet läggas till i historiken
2. historiken sparas i `localStorage`
3. historiken på sidan uppdateras direkt

Användaren ska alltså **inte behöva ladda om webbsidan** för att det nya studentkortet ska visas i historiken.

Det **senast skapade studentkortet ska visas överst**.

Historikposterna ska skapas och läggas till i DOM med JavaScript.

Rensa formuläret

När användaren klickar på knappen **Rensa** ska:

- formulärets fält återställas
- eventuella felmeddelanden tas bort

Den sparade historiken ska **inte** påverkas av denna knapp.

Radera historik

Användaren ska kunna radera hela den sparade historiken.

När användaren klickar på **Radera historik** ska:

- historiken tas bort från `localStorage`
- den array som används för historiken tömmas
- den visade historiken på sidan uppdateras direkt

Användaren ska inte behöva ladda om sidan för att se förändringen.

Strukturera programmet

I `main.js` finns en påbörjad struktur med förslag på funktioner som kan användas för att dela upp programmet i mindre delar.

Funktionerna har olika ansvarsområden, exempelvis att:

- validera formuläret
- visa felmeddelanden
- skapa studentkort
- spara historik
- läsa in historik
- visa historik
- rensa formuläret
- radera historik

Du ska implementera funktionaliteten i dessa funktioner och koppla samman programmets olika delar med lämpliga eventlyssnare.

Försök låta varje funktion utföra en tydlig uppgift och återanvänd befintliga funktioner där det är lämpligt.

Din programkod ska vara **väl kommenterad och läsbar**. Kommentarerna ska hjälpa till att förklara vad olika delar av programmet gör. Använd även tydliga och beskrivande namn på variabler och funktioner.

Tips och ledtrådar

Detta är en större programmeringsuppgift där flera olika delar behöver fungera tillsammans. Det är därför en bra idé att arbeta med **en del i taget och testa ofta**.

Försök exempelvis först få formuläret och studentkortet att fungera innan du börjar arbeta med historiken och `localStorage`.

På sidan **Tips och ledtrådar till Laboration 5 (https://canvas.miun.se/courses/11718/pages/tips-och-ledtradar-till-laboration-5)** finns ytterligare hjälp med hur du kan angripa de olika delarna av uppgiften samt mindre kodexempel.

Försök gärna lösa respektive del själv först och använd sedan ledtrådarna om du fastnar.

Krav för godkänt

Laborationen bedöms med **Godkänd (G)** eller **Underkänd med möjlighet till komplettering (Fx)**.

För att laborationen ska anses vara godkänd krävs att:

- Applikationen är färdigställd och fungerar enligt uppgiftsbeskrivningen ovan.

- JavaScript används för att hantera formuläret, validera användarens inmatning och skapa studentkortet dynamiskt.
- Tydliga felmeddelanden visas vid felaktig inmatning och ingen information sparas om valideringen misslyckas.
- Skapade studentkort hanteras som objekt i en array och historiken sparas och hämtas med `localStorage`, `JSON.stringify()` och `JSON.parse()`.
- Historiken uppdateras dynamiskt och funktionaliteten för att rensa formulär respektive historik fungerar enligt beskrivningen.
- JavaScript-koden är väl kommenterad, läsbar och strukturerad i lämpliga funktioner med tydliga ansvarsområden.
- Git har använts under hela arbetet och repositoryts versionshistorik innehåller **regelbundna commits med tydliga och beskrivande commit-meddelanden**.

Publicering

När laborationen är färdig ska webbplatsen publiceras på en publikt tillgänglig webbhost.

Använd någon av de publiceringsmetoder eller plattformar som du arbetat med tidigare i kursen.

Testa alltid den **publicerade versionen** innan du lämnar in laborationen och kontrollera bland annat att:

- studentkort kan genereras
- valideringen fungerar
- historiken sparas och visas
- historiken finns kvar efter att sidan laddats om
- historiken uppdateras direkt när ett nytt kort skapas
- historiken kan raderas

Inlämning

Vid inlämning ska du lämna:

- en länk till den publicerade webbplatsen
- en länk till ditt GitHub-repository

Kontrollera att båda länkarna fungerar innan du skickar in laborationen.

Repositoryt ska innehålla samtliga filer för laborationen samt en versionshistorik som visar att Git har använts regelbundet under arbetets gång.

Kom ihåg att göra Github-repot publikt under rättningstiden.

Visa matris

Moment 5			
Kriterier	**Bedömningar**		**Poäng**
Studentkortsgeneratorn är implementerad och fungerar enligt instruktionerna.	**Uppfylld** **1 poäng**	**Ej uppfylld** **0 poäng**	/1 poäng

Kriterier	Bedömningar		Poäng
Formuläret valideras enligt instruktionerna och tydliga felmeddelanden visas vid felaktig inmatning.	**Uppfylld** **1 poäng**	**Ej uppfylld** **0 poäng**	/1 poäng
Historiken fungerar enligt instruktionerna och lagras med localStorage och JSON.	**Uppfylld** **1 poäng**	**Ej uppfylld** **0 poäng**	/1 poäng
JavaScript-koden är läsbar, kommenterad och uppdelad i funktioner med tydliga ansvarsområden.	**Uppfylld** **1 poäng**	**Ej uppfylld** **0 poäng**	/1 poäng
GitHub-repot visar en tydlig utvecklingsprocess med regelbundna och beskrivande commits.	**Uppfylld** **1 poäng**	**Ej uppfylld** **0 poäng**	/1 poäng
Applikationen är publicerad och fungerar via den inlämnade länken.	**Uppfylld** **1 poäng**	**Ej uppfylld** **0 poäng**	/1 poäng

Redigera Visa Infoga Format Verktyg Tabell

12pt Paragraf

p 0 ord

(https://canvas.miun.se/courses/11718/modules/items/231319)

(https://canvas.miun.se/courses/11718/modules/items/250